Ananya's Documentation Writing TODO: qkrylov

Updated: 27 Sep 2026

The Big Picture: Read This First

qkrylov is a polyglot (C++, Python, Julia) quantum many-body physics engine. Your job is to write the narrative documentation.

Where files actually live

The PDF had wrong paths. The docs source lives entirely inside:

```
website/src/content/docs/
```

Everything below refers to paths inside this folder. The `docs/` folder at the repo root contains only auto-generated C++ API reference output — **do not touch it**.

How the docs system works

The docs system is **Astro + Starlight** (not Zensical — that was the old system). The sidebar nav is configured in `website/astro.config.mjs`.

To preview your work locally, from the repo root:

```
cd website
npm run dev
```

Then open `http://localhost:4321`. Hot-reload is automatic — every save triggers a browser refresh.

Two Modes of Writing Documentation

Mode 1 — Plain Markdown (reference pages, theory, concept explanations)

Write a `.md` file anywhere under `website/src/content/docs/`. Starlight auto-routes it. It will appear at a URL matching the path.

Example: `website/src/content/docs/theory/lanczos_memory.md` is already in the sidebar because Aritra added this entry to the `sidebar` array in `astro.config.mjs`:

```
{ label: 'Single-Pass vs Two-Pass Lanczos', link: '/theory/lanczos_memory/' },
```

You create the file. You tell Aritra the path so he can add it to the sidebar. Done.

Mode 2 — Polyglot Tutorial (live code demos in Python, Julia, and C++)

Instead of a `.md` file, create a **folder** with three source files:

```
website/src/content/docs/getting_started/tutorial_something/
├── main.ipynb   ← you write this (Python narrative + code)
├── main.jl      ← you write this (same logic in Julia)
└── main.cpp     ← you write this (same logic in C++)
```

A Vite plugin (`website/plugins/polyglot-vite-plugin.mjs`) automatically transforms `main.ipynb` into an MDX page at build/dev time, stitching in the Julia and C++ blocks as synchronized language tabs.

After creating the folder, tell Aritra. He will:

1. Create a thin `.astro` wrapper page at `website/src/pages/getting_started/tutorial_something.astro` that imports the notebook (look at `tutorial_hamiltonian.astro` as the template).
2. Add a sidebar entry to `astro.config.mjs`.

Do NOT create any `.astro` file or edit `astro.config.mjs` yourself.

Reference example: study `website/src/content/docs/getting_started/tutorial_hamiltonian/` — it has the three files you need to model yours on.

How IDs work across the three files

The plugin matches C++/Julia code blocks to Python cells by ID annotations. **IDs must be identical** across all three files.

In `main.ipynb` Python cells — the first line of each code cell is read as the ID marker:

```
# [ID: init_basis]
basis = SpinHalfBasis(N=16, sz=0)
```

```
site  = SpinHalfSite()
```

In `main.jl`:

```
# [ID: init_basis]
b = SpinHalfBasis(16, Sector(0))
# [END: init_basis]
```

In `main.cpp`:

```
// [ID: init_basis]
qkrylov::basis::SpinHalf b(16, qkrylov::sector::Sz(0));
// [END: init_basis]
```

If a step has no meaningful C++ or Julia equivalent, simply omit it from those files.

The Golden API References (Ground Truth)

Never invent function names. Always look here first:

Language	Path
Python	`docs/src/content/docs/api/auto/python/`
Julia	`docs/src/content/docs/api/auto/julia/`
C++	`docs/src/content/docs/api/auto/cpp/`

> **Critical note for Julia:** `SinglePass` was renamed to `OnePass`. Always write `Lanczos(variation=OnePass())` or simply `Lanczos()` (defaults to `OnePass`). Never write `SinglePass`.

> **Do NOT edit anything in `docs/src/content/docs/api/auto/`** — those are Pritipriya's auto-generated authoritative references. Leave them alone.

🔴 High Priority — Pages in the Nav with Broken Content

These are already linked in the sidebar. Users can click them right now and see wrong code. Fix these first.

1. `website/src/content/docs/getting_started/ecosystem.md`

Current state: Uses completely wrong API. Wrong import paths, nonexistent methods (`solver.get_ground_state()`, `H.to_scipy_sparse()`).

What it should be: A practical page showing how to use qkrylov output downstream. Use the 1D Heisenberg model (L=10, Sz=0 sector) as the running example throughout. Cover these four things with minimal working code snippets:

1. **Convert eigenvector to NumPy** — `result.eigenvector` is already a `np.ndarray`; show this explicitly with a print/shape check.
2. **Export Hamiltonian as a SciPy CSR matrix** — use `H.to_sparse()` (returns `scipy.sparse.csr_matrix`).
3. **Wrap as a `scipy.sparse.linalg.LinearOperator`** — use `H.aslinearoperator()`, then pass to `scipy.sparse.linalg.eigsh` as a cross-check.
4. **Export eigenvector as a `torch.Tensor`** — `torch.from_numpy(result.eigenvector)`.

Reference: `docs/src/content/docs/api/auto/python/hamiltonian.md` — `MatrixFreeHamiltonian` methods section (`aslinearoperator()`, `to_sparse()`).

This is **Mode 1** (plain markdown, no notebook needed).

---

2. `website/src/content/docs/getting_started/sciml_julia.md`

Current state: Uses `SpinBasis`, `EigenProblem`, `build_heisenberg_hamiltonian` — none of these exist in the actual API.

What it should be: A Julia-focused page showing the SciML `solve(prob, alg)` pattern. Cover:

- `GroundStateProblem` + `solve(prob, Lanczos())` — ground state energy and eigenvector
- `ExcitedStatesProblem` + `solve(prob, Davidson(n_eig=3))` — lowest 3 eigenstates
- `ThermalProblem` + `solve(prob, FTLM())` — finite-temperature observables at a single β
- The decoupled workflow: `ftlm_sample(H; n_random=50, n_steps=100)` → `FTLMSamples` handle → `ftlm_evaluate_sweep(samples, betas)` — zero extra SpMV for any temperature grid

Reference: `docs/src/content/docs/api/auto/julia/solvers.md` (full signatures for `ftlm_sample`, `ftlm_evaluate_sweep`, `FTLMSamples`) and

`docs/src/content/docs/api/auto/julia/hamiltonian.md` (for `MatrixFreeHamiltonian` constructor).

This is **Mode 1** (plain markdown, no notebook needed).

---

3. `website/src/content/docs/api/index.md`

Current state: Stub. Says "Under Construction."

What it should be: A short (~200 word) overview of the four API layers and when to use each, ending with a link table. Good warmup task.

Layer	When to use
Python	Interactive research, Jupyter notebooks, scripting
Julia	SciML ecosystem, DifferentialEquations.jl integration
C++	Maximum performance, custom kernels, embedding in other C++ projects
C ABI	Language-agnostic FFI — use when writing bindings for any other language

Note: this file is at `website/src/content/docs/api/index.md` (not `docs/src/...` — that is the auto-gen folder).

This is **Mode 1**.

🟡 Medium Priority — AI-Written Pages That Need Human Physics Check

These pages exist and have plausible-looking prose but have **not been reviewed** by a human physicist. Check math and code examples against the API refs and fix anything wrong.

4. `website/src/content/docs/theory/matrix_free_spmv.md`

Explains why we never build the full Hamiltonian matrix. Check the memory scaling claim (should be $O(N \cdot \text{nnz_per_row})$ for CSR vs $O(N)$ for matrix-free state vector) and verify Python code examples use real API names.

5. `website/src/content/docs/theory/lanczos_memory.md`

Explains Single-Pass ($O(k \times N)$ memory, stores full Krylov subspace) vs Two-Pass ($O(N)$ memory, recomputes vectors on the fly at the cost of one extra SpMV per step). Verify the memory formulas are correct and make sure it recommends `LanczosTwoPass` for L > 22 on GPU.

6. `website/src/content/docs/theory/precision_stability.md`

Explains FP32 vs FP64 in Lanczos. Key physics to verify:

- FP32 is faster per iteration but loss of orthogonality accumulates sooner → "ghost states" (spurious duplicate eigenvalues in the Ritz spectrum)
- FP64 maintains cleaner orthogonality → fewer iterations to convergence
- With early stopping, FP64 often wins in wall-clock time despite slower individual iterations

7. `website/src/content/docs/replications.md`

Contains paper replications (Bethe Ansatz for 1D Heisenberg, etc.). Check every Python code example uses the correct current API function names. Fix any wrong calls.

🟢 New Tutorials to Write (Not Started)

All use **Mode 2** (Polyglot Tutorial). Create the folder with `main.ipynb`, `main.jl`, and `main.cpp`, then tell Aritra — he handles the `.astro` wrapper and sidebar wiring.

8. Tutorial: FTLM (Finite Temperature Lanczos Method)

Create: `website/src/content/docs/getting_started/tutorial_ftlm/`

Storyline:

1. Build the Heisenberg Hamiltonian (can reference the Hamiltonian tutorial).
2. Use the decoupled workflow: `FTLM(mode="cached").sample(H)` → `FTLMSamples` (Stage 1, heavy SpMV, run once).
3. Call `samples.evaluate_sweep(betas)` on a temperature grid (Stage 2, zero SpMV cost, instant).
4. Plot $E(\beta)$, $C_v(\beta)$, $S(\beta)$ vs temperature.
5. Show the decoupled advantage explicitly: change the temperature grid and re-run Stage 2 at zero cost.

Reference: `docs/src/content/docs/api/auto/python/solvers.md` — `FTLM`, `FTLMSamples`, `ftlm_sample`, `ftlm_evaluate_sweep`.

---

9. Tutorial: Quantum Dynamics & Spectral Functions

Create: `website/src/content/docs/getting_started/tutorial_dynamics/`

Storyline:

1. Build Hamiltonian, get ground state via `Lanczos().solve(H)` → `result.energy`, `result.eigenvector`.
2. Time evolution: `TimeEvolve(time_grid=times).solve(H, psi0)` — compute survival probability $|\langle\psi_0|\psi(t)\rangle|^2$ from `result.survival_probabilities`.
3. Spectral function: `CorrectionVector(e0=result.energy, omega=…, eta=0.05).solve(H)` at a grid of ω values.
4. Show a simple spectral function plot.

Reference: `docs/src/content/docs/api/auto/python/solvers.md` — `TimeEvolve`, `CorrectionVector`, `FTLMDynamics`.

---

10. Tutorial: Excited States (Davidson Solver)

Create: `website/src/content/docs/getting_started/tutorial_excited_states/`

Storyline:

1. Build Hamiltonian.
2. Use `Davidson(n_eig=5).solve(H)` → `DavidsonResult` with `.eigenvalues` list and `.eigenvectors` list.
3. Explain why Davidson is better than running Lanczos 5 separate times (deflation: each new eigenpair is found orthogonal to already-converged ones, avoiding re-convergence of lower states).
4. Show an energy level spectrum plot.

Reference: `docs/src/content/docs/api/auto/python/solvers.md` — `Davidson`, `DavidsonResult`.

Writing Style Guide

- **Use the Heisenberg model everywhere.** Canonical example: L=10 sites, J=1.0, Sz=0 sector.
- **Never invent function names.** Copy them exactly from the API reference files under `docs/src/content/docs/api/auto/`.
- **Write prose, not bullets.** Tutorials should read like a lab notebook — explain *why*, not just *how*.
- **Math is encouraged.** Use LaTeX: inline `$ … $` and display `$$ … $$` freely. KaTeX renders it via the Astro rehype plugin.

- **Tabs for polyglot snippets in Mode 1 pages.** Use the remark-tabs directive syntax (Mode 2 pages generate tabs automatically from the `.ipynb` / `.jl` / `.cpp` files):

  ````
  ::: tabs
  == Python
  ```python
  # your code here
  ```

  == Julia

  ```
  # your code here
  ```

  == C++

  ```
  // your code here
  ```

  :::
  ````

What NOT to Do

- **Do not edit files in `docs/src/content/docs/api/auto/`** — auto-generated, owned by Pritipriya.
- **Do not create `.astro` files or edit `astro.config.mjs`** — Aritra handles that.
- **Do not touch `website/dist/`, `website/node_modules/`, `website/public/.generated_webp/`** — build artifacts.
- **Do not confuse `docs/` with `website/src/content/docs/`** — the former is C++ API auto-gen output only. You work in the latter.